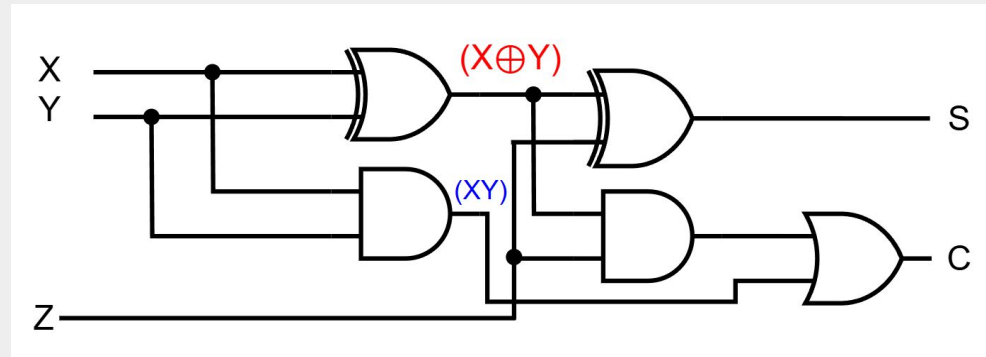


Lecture 6

Adder-Subtractor with Applications



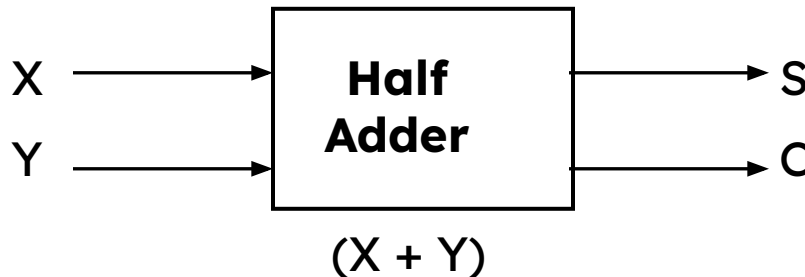
CSE260 : Digital Logic Design

Half Adder

- The combinational circuit that performs the addition of two bits is called a half-adder.

If the bits are X and Y, we will get C (Carry) & S (Sum)

$$\begin{array}{r} X \\ + Y \\ \hline C \quad S \end{array}$$





Half Adder

Now, we draw the truth table for all possible input combinations

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \quad 0 \end{array} \qquad \begin{array}{r} 0 \\ + 1 \\ \hline 0 \quad 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 0 \quad 1 \end{array} \qquad \begin{array}{r} 0 \\ + 0 \\ \hline 1 \quad 0 \end{array}$$

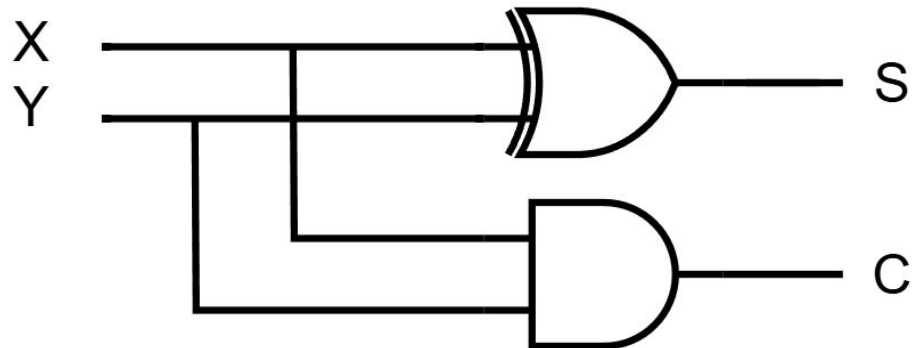
X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Half Adder

From the truth table, we find out the SOP to get the boolean function & simplify it.

X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Now, we draw the logic diagram.



$$C = XY$$

$$S = X'Y + XY' = X \oplus Y$$

Full Adder

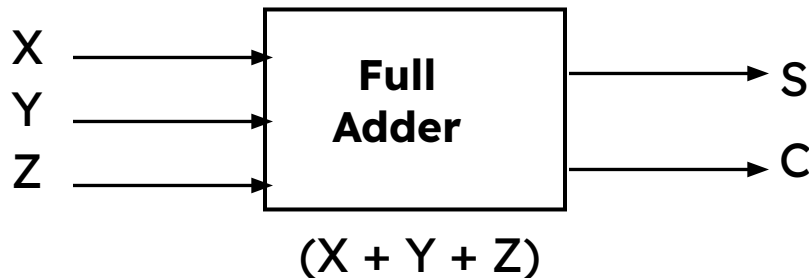
Half-adder adds up only two bits.

To add two binary numbers, we need to add 3 bits (including the carry).

Example:

		1	1	1	carry
		0	0	1	1
		X			
+		0	1	1	1
		Y			
		1	0	1	0
		S			

Need **Full Adder** (so called as it can be made from two half-adders).



Full Adder

From the truth table, we find out the SOP to get the boolean function & simplify it.

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Note:

Z - carry in (to the current position)
C - carry out (to the next position)

$$C = X'YZ + XY'Z + XYZ' + XYZ$$

$$S = X'Y'Z + XYZ + X'YZ' + XY'Z'$$

Full Adder

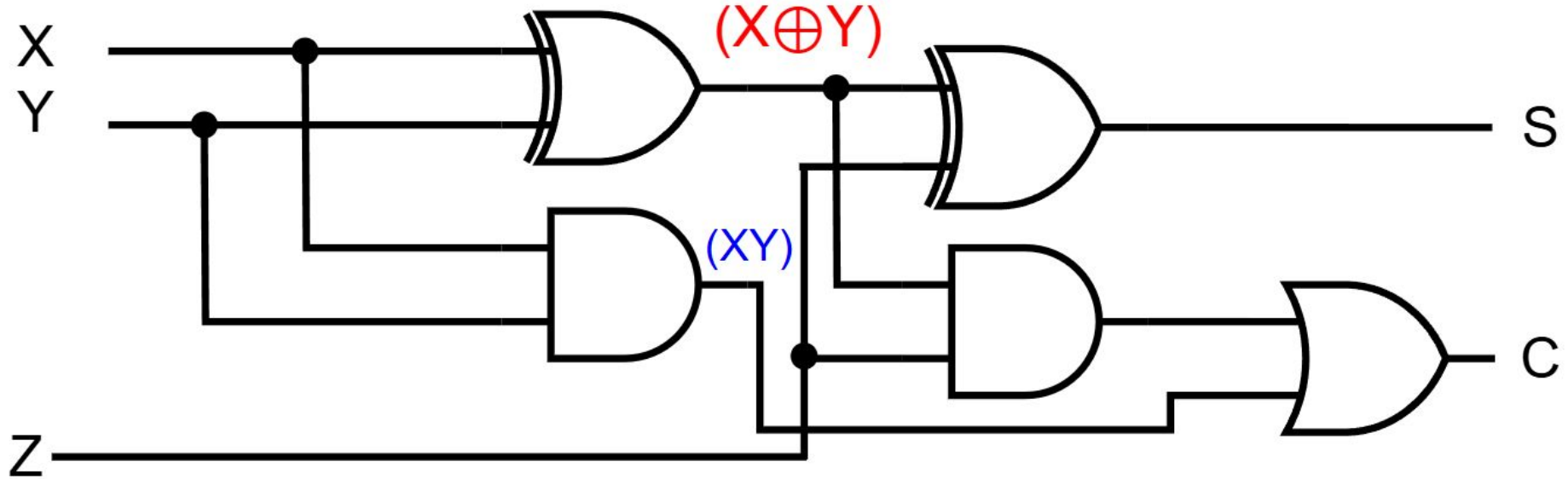
Simplifying,

$$\begin{aligned}
 C &= X'YZ + XY'Z + XYZ' + XYZ \\
 &= Z (X'Y + XY') + XY (Z + Z') \\
 &= Z (X \oplus Y) + XY
 \end{aligned}$$

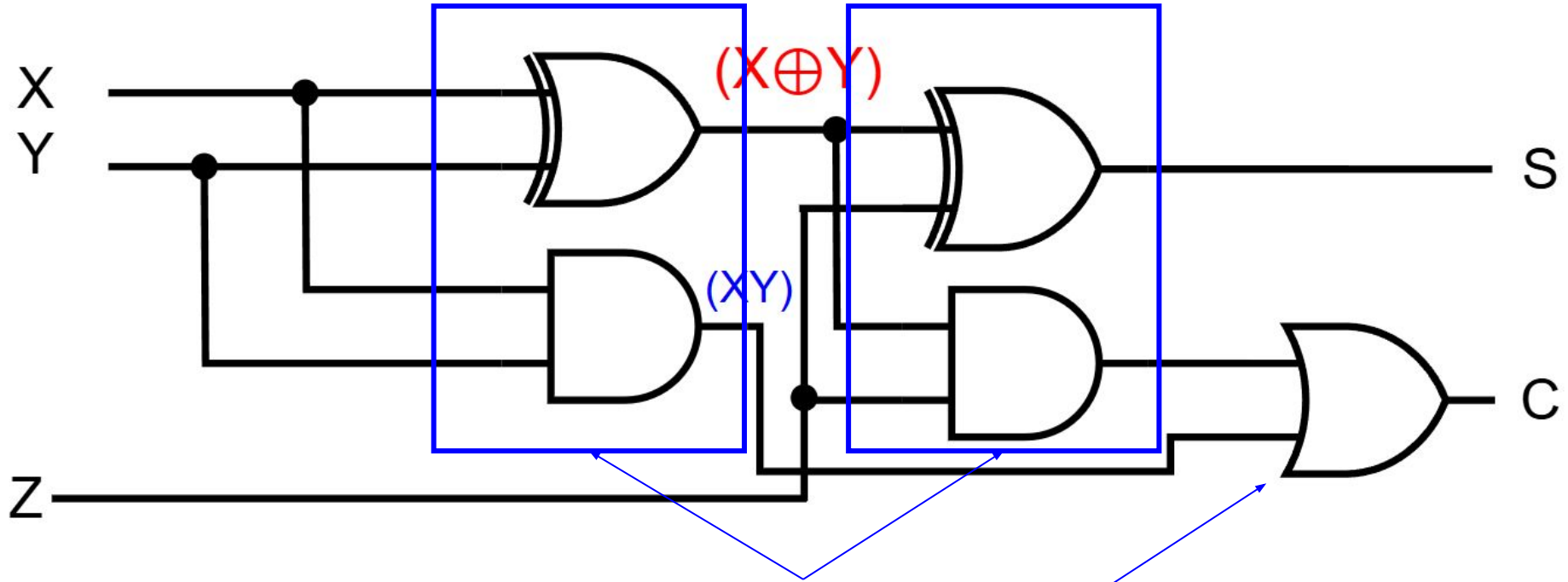
$$\begin{aligned}
 S &= X'Y'Z + X'YZ' + XY'Z' + XYZ \\
 &= X' (Y'Z + YZ') + X (Y'Z' + YZ) \\
 &= X' (Y \oplus Z) + X (Y \oplus Z)' [(Y \oplus Z) = A] \\
 &= X'A + XA' \\
 &= X \oplus A \\
 &= X \oplus Y \oplus Z
 \end{aligned}$$

Full Adder

Circuit



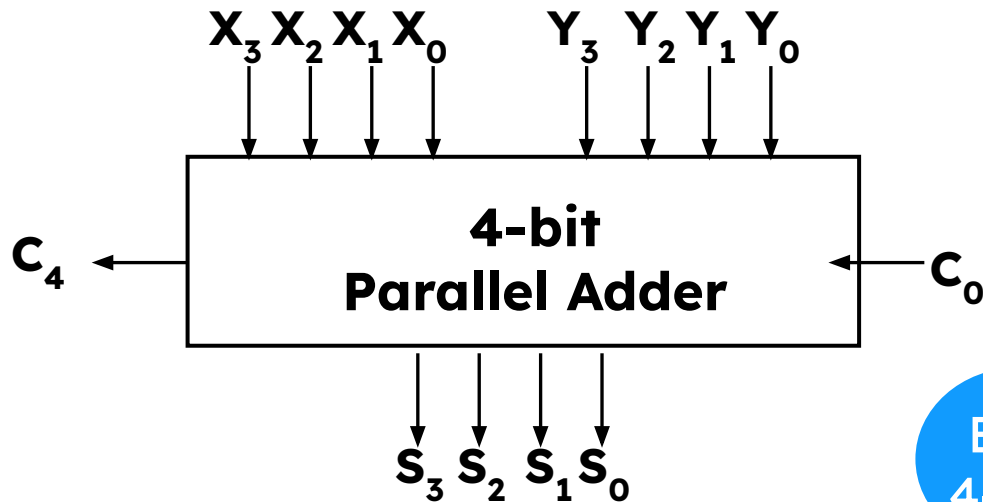
Why It Is Called Full Adder?



Full Adder made from two Half-Adders (+ OR gate).

4-bit Parallel Adder

Consider a circuit to add two 4-bit numbers together and a carry-in, to produce a 5-bit result:



Black-box view of
4-bit parallel adder

5-bit result is sufficient because the largest result is:

$$(1111)_2 + (1111)_2 + (1)_2 = (11111)_2$$

4-bit Parallel Adder

SSI design technique should not be used.

Truth table for 9 inputs very big, i.e. $2^9=512$ entries:

$X_3X_2X_1X_0$	$Y_3Y_2Y_1Y_0$	C_0	C_4	$S_3S_2S_1S_0$
0 0 0 0	0 0 0 0	0	0	0 0 0 0
0 0 0 0	0 0 0 0	1	0	0 0 0 1
0 0 0 0	0 0 0 1	0	0	0 0 0 1
...	...	...	...	...
0 1 0 1	1 1 0 1	1	1	0 0 1 1
...	...	...	...	...
1 1 1 1	1 1 1 1	1	1	1 1 1 1

Simplification very complicated as no. of input is 9 so $512(=2^9)$ possible combination will be there in truth table!

4-bit Parallel Adder

- Alternative design possible.
- Addition formula for each pair of bits (with carry in),

$$C_{i+1}S_i = X_i + Y_i + C_i$$

has the same function as a full adder.

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

$$S_i = X_i \oplus Y_i \oplus C_i$$

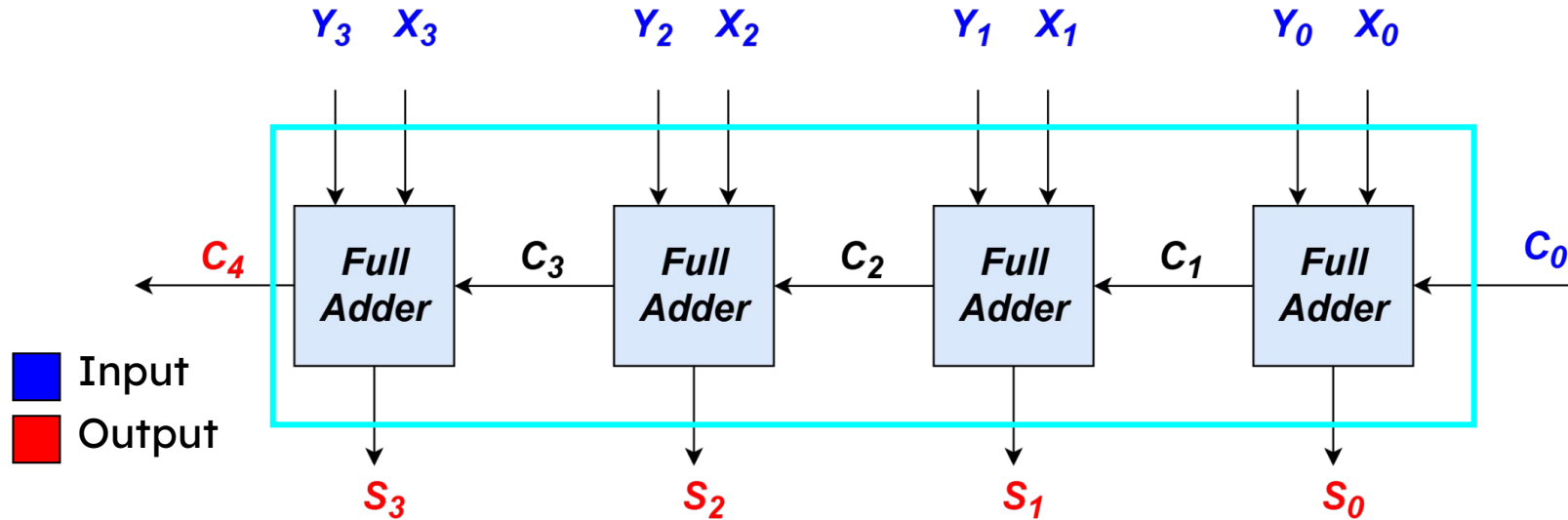
NB: Similar to FA equation

$$C = XY + (X \oplus Y)Z$$

$$S = (X \oplus Y) \oplus Z$$

4-bit Parallel Adder

Cascading 4 full adders via their carries, we get:



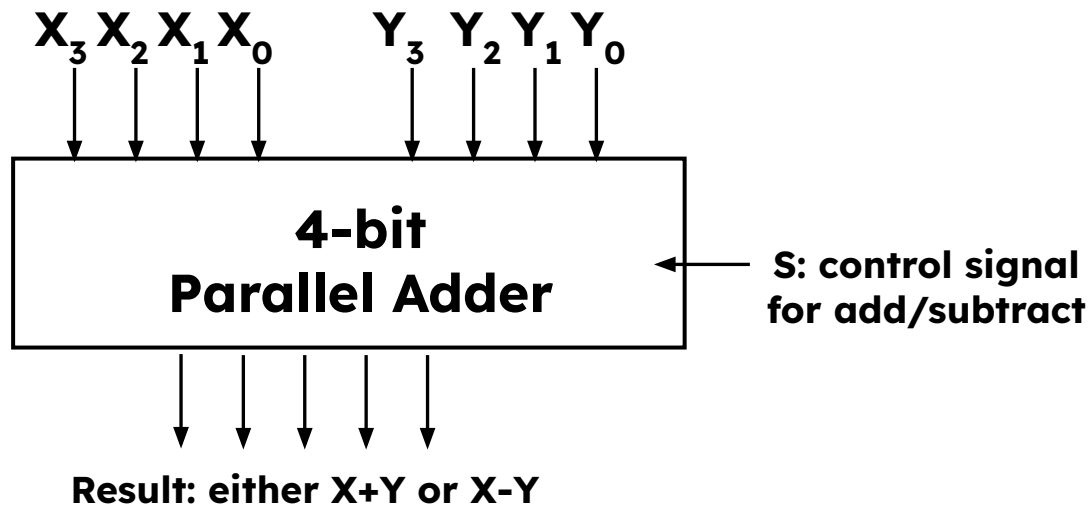
n-bit full adder requires n full- adders

Examples

- Simple examples using 4-bit parallel adder as building blocks:
 - ❖ Adder cum Subtractor
 - ❖ 16-Bit Parallel Adder
 - ❖ BCD to excess 3 code converter

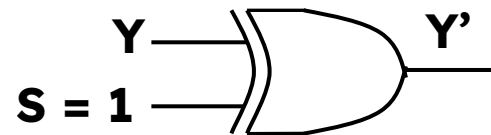
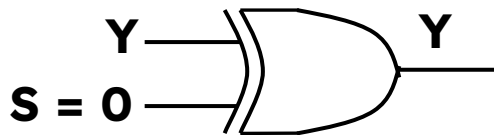
4-bit Parallel Adder-Subtractor

- Subtraction can be performed through addition using 2s-complement numbers.
- Hence, we can design a circuit which can perform **both addition and subtraction**, using a parallel adder.



4-bit Parallel Adder-Subtractor

- Design requires:
 - (i) **XOR gates:**



such that: output = Y when $S=0$
 = Y' when $S=1$

- (ii) **S connected to carry-in.**

S	Y	Output
0	0	0
0	1	1
1	0	1
1	1	0

4-bit Parallel Adder-Subtractor

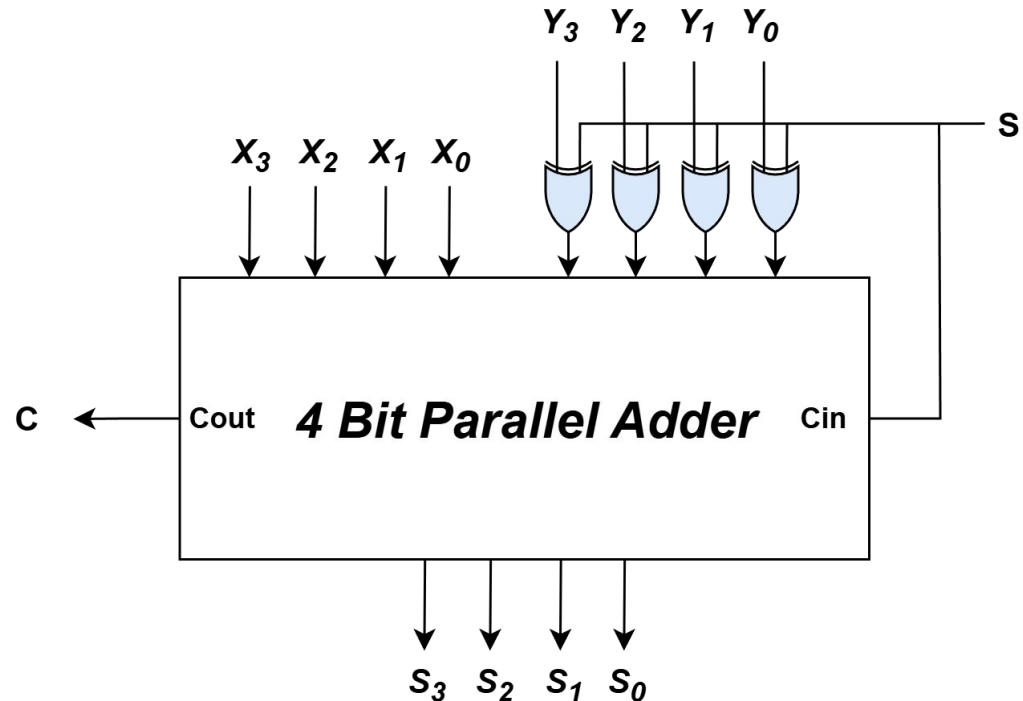
Adder cum subtractor circuit:

Analysis:

If $S=1$, then

$X + (1\text{'s complement of } Y) + 1$
appears as the result.

If $S=0$, then $X+Y$ appears as
the result.



Example

Applications: Condition Based Adder Subtractor

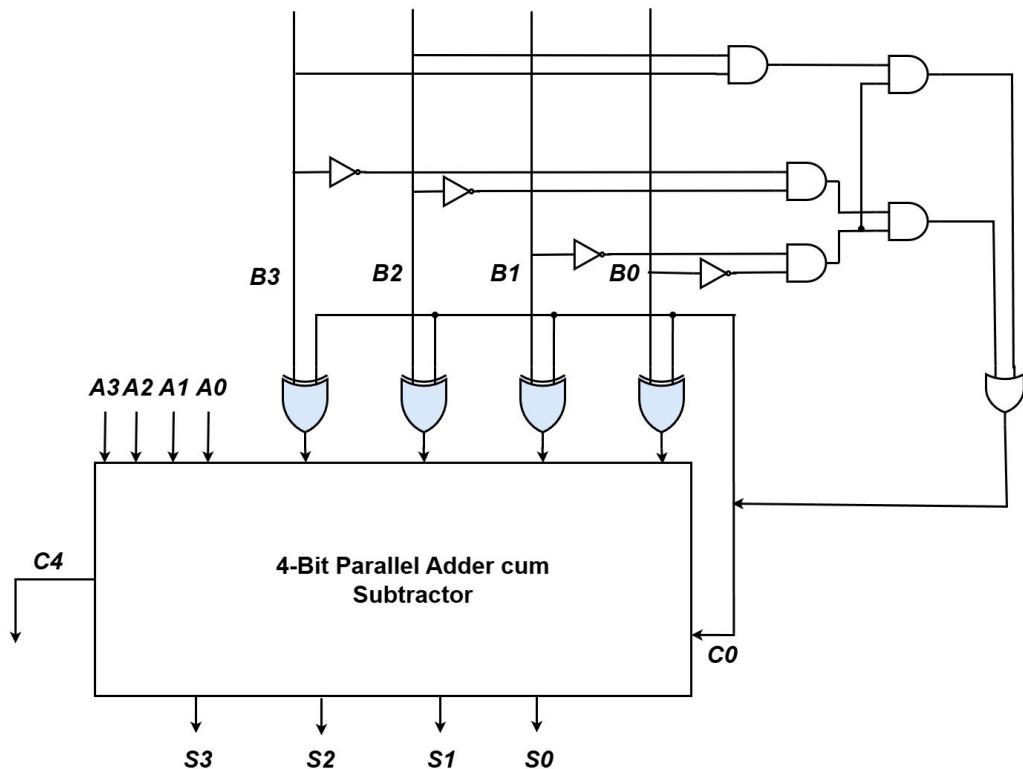
Design a circuit that takes two binary number A, B and performs $A-B$, if certain condition is met or else perform $A+B$

Two Methods

- Brute Force Approach: Find the minterms and use the SOP(s) directly
- Smart Approach: Find existing patterns within the SOP(s)

Example

Perform A-B, if B is Divisible by 12 using Brute Force Approach



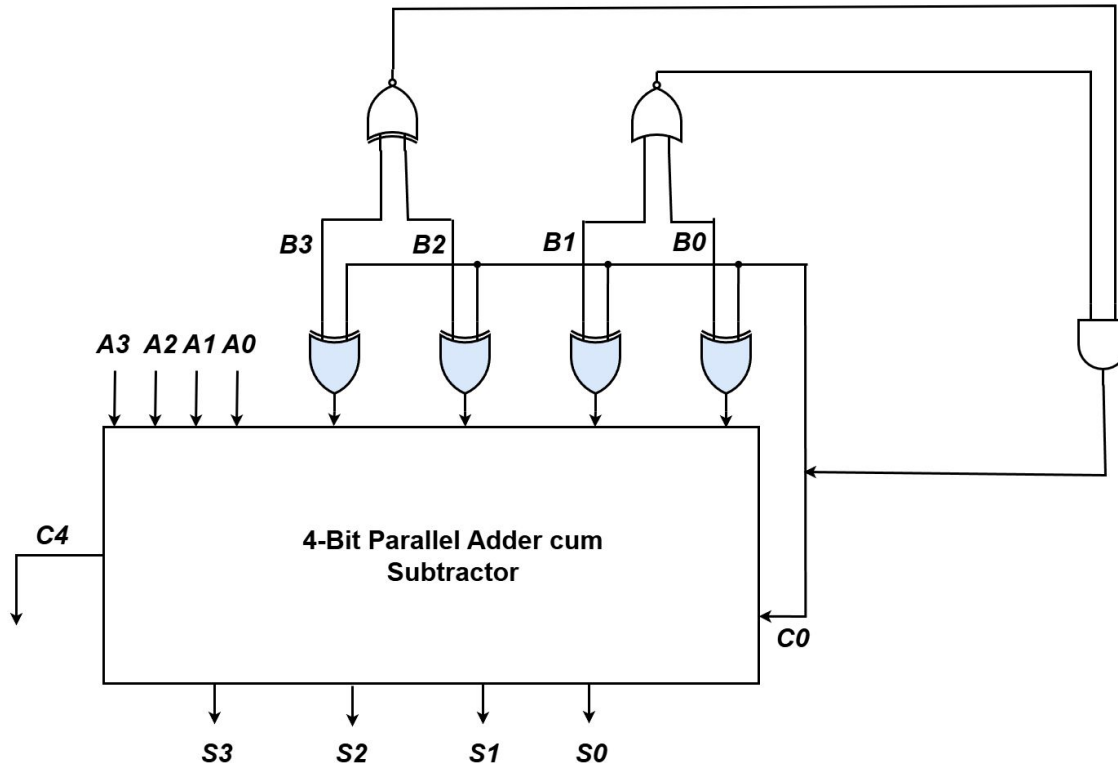
Minterm	Product
0	$B_3'B_2'B_1'B_0'$
12	$B_3B_2B_1'B_0'$

- Only the numbers 0 and 12 are divisible by 12 among all 4-bit numbers, So the following expression will solve the problem,

$$F(B_3, B_2, B_1, B_0) = B_3B_2B_1'B_0' + B_3'B_2'B_1'B_0'$$
- If the number of minterm increases, you may use KMAP to simplify the circuit.

Example

Perform $A-B$, if B is Divisible by 12 using Patterns

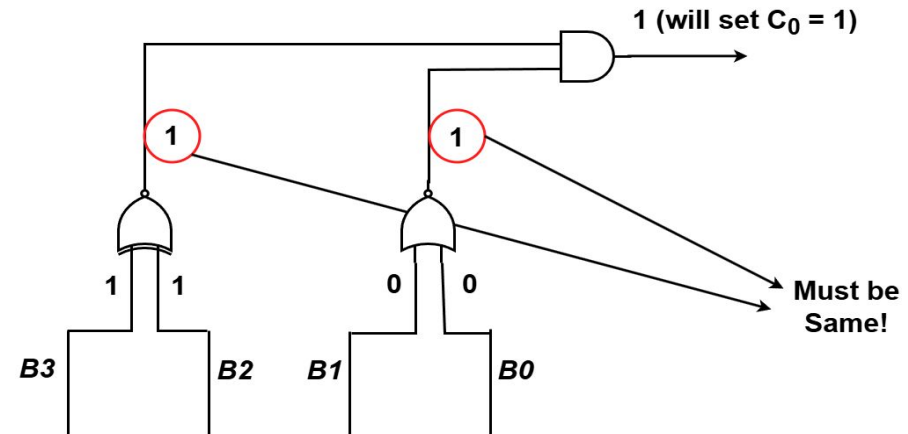
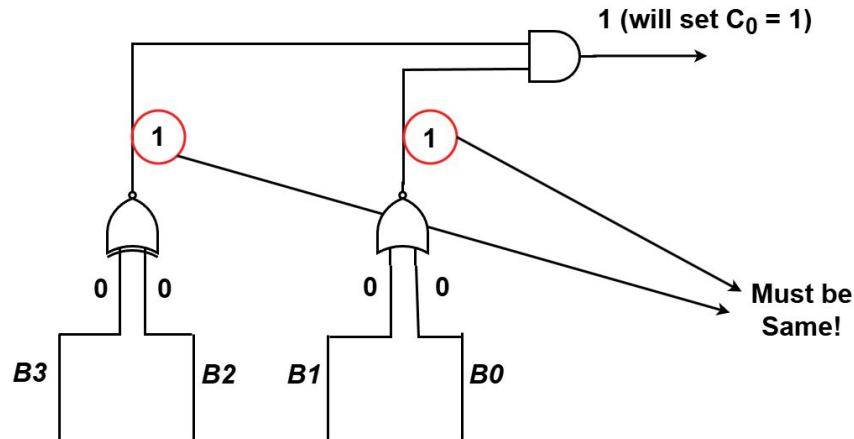


Possible combinations among B_3 and B_2 are 00 and 11, hence XNOR gate was used to combine them as only XNOR gate will provide 1 for these two unique combinations. NOR gate was used for latter variables for the same reason.

Finally, combining them with AND gate ensures the output of the AND gate will only be 1 if the current combination is indeed divisible by 12. If C_0 receives 1 then it will perform $A-B$.

Example

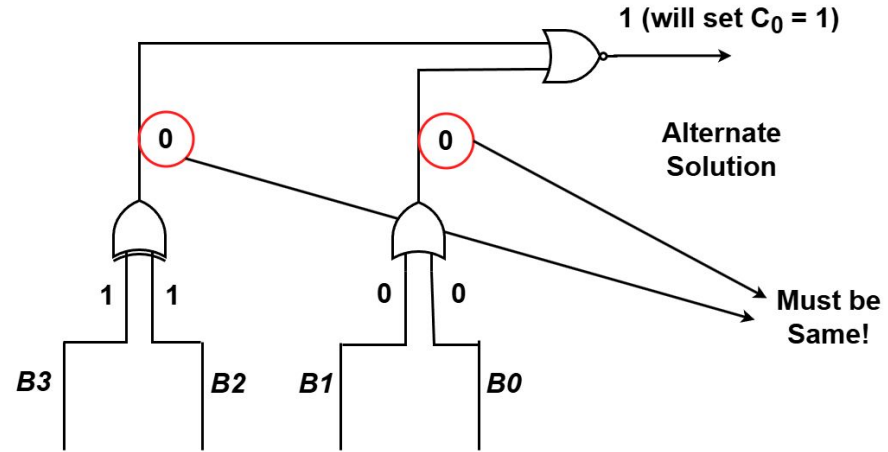
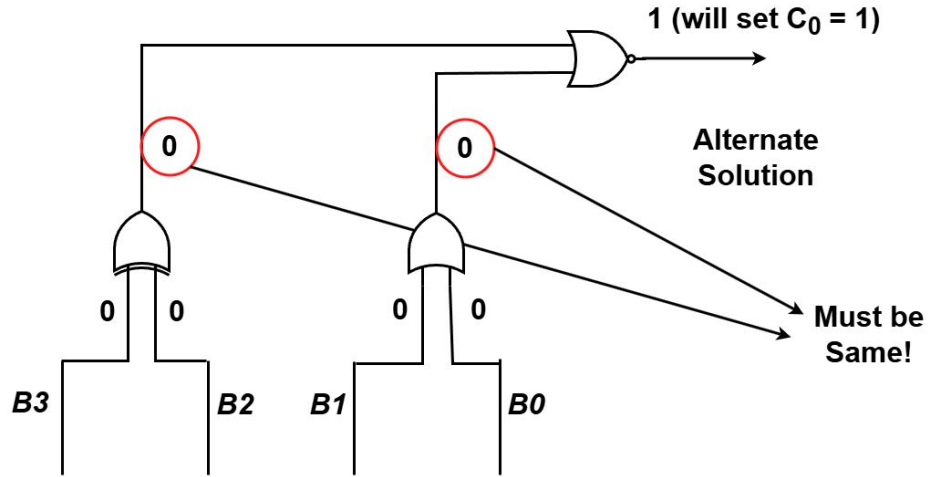
Perform A-B, if B is Divisible by 12 using Patterns (Cont)



The solution is simulated for both valid combinations who are divisible by 12, 0000 and 1100. The red circled elements indicate that the output of the intermediary gates cannot mismatch, any mismatch in those circled outputs means that the current combination is not divisible by 12 and the final output will give 0 which will result in C_0 being 0 hence, performing A+B likewise

Example

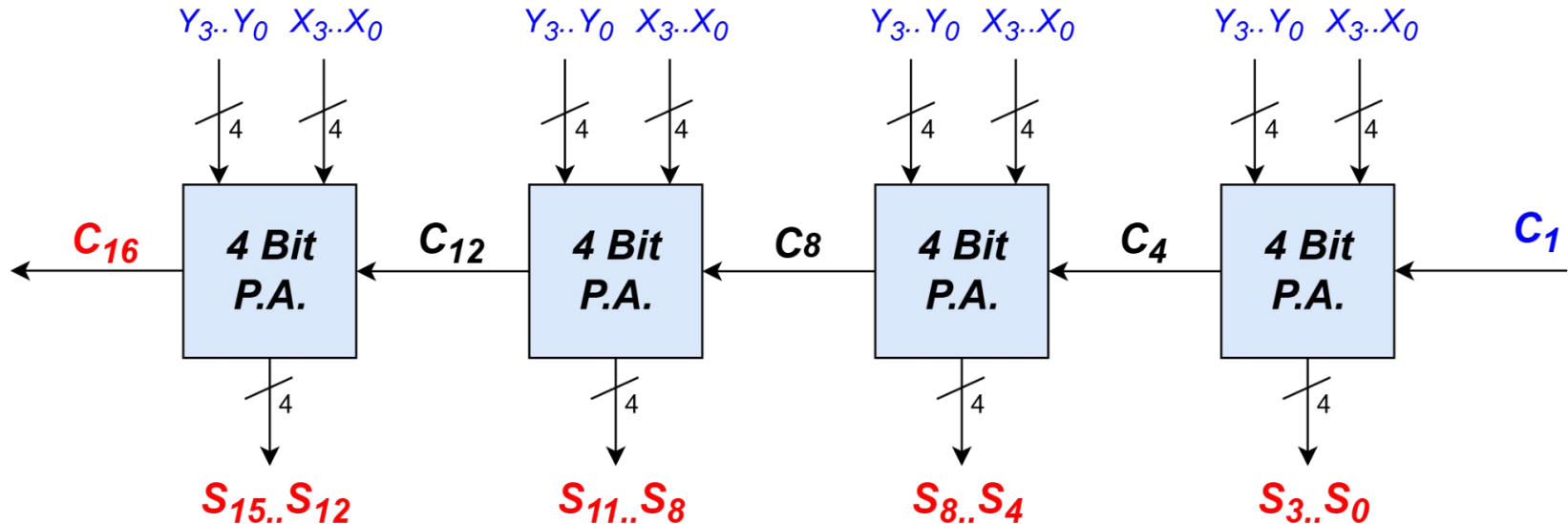
Perform $A - B$, if B is Divisible by 12 using Patterns (Alternate Solution)



The alternate solution using different logic gates also fulfill the same logic

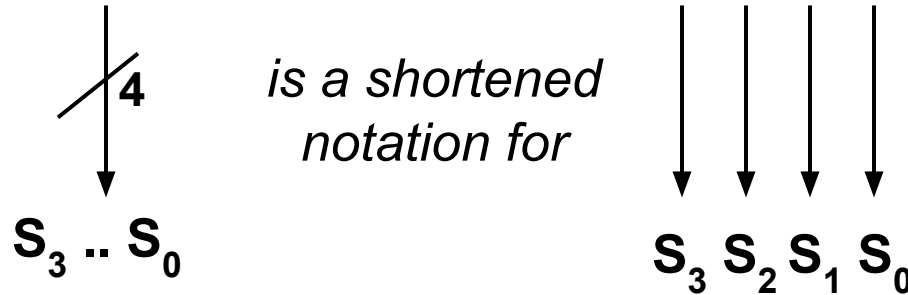
16-bit Parallel Adder

- Larger parallel adders can be built from smaller ones.
- Example: a **16-bit parallel adder** can be constructed from four 4-bit parallel adders:



16-bit Parallel Adder

- Shortened notation for multiple lines.

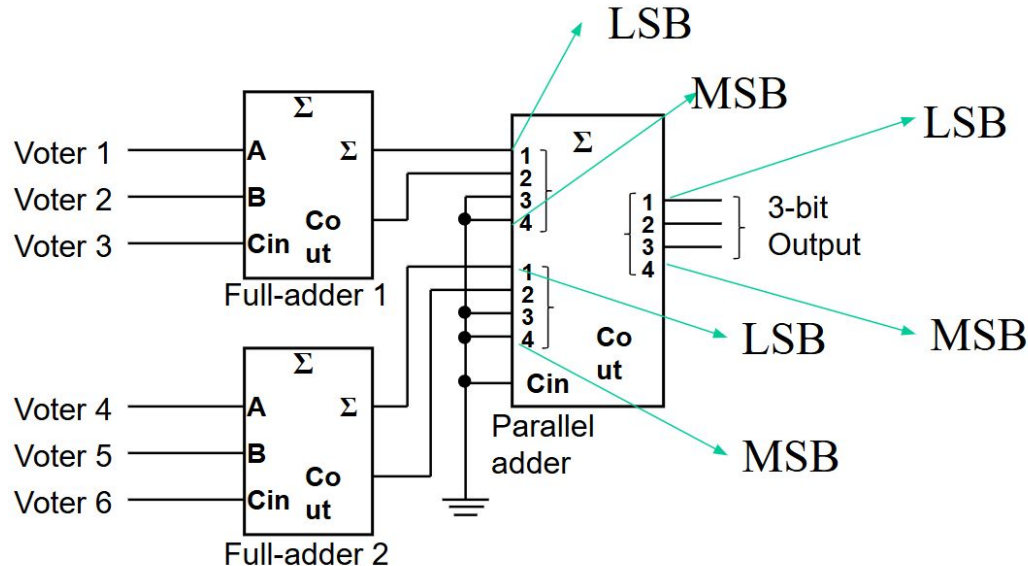


16-bit parallel adder ripples carry from one 4-bit block to the next. Such ripple-carry circuits are “slow” because of long delays needed to propagate the carries.

Arithmetic Circuits: Cascading Adders

Application: 6-person voting system.

- Use FAs and a 4-bit binary parallel adder.
- Each FA can sum up to 3 votes.



Try it yourself

Example: Using Full adders and 4 Bit Parallel Adder,

1. Design a 12 person voting system
2. Design a 13 person voting system, your circuit must be efficient.
3. Design a 14 person voting system

Using Half Adder and 4 Bit Parallel Adder

1. Design 10 person voting system. Your circuit must be efficient

The diagram illustrates a 4-bit majority logic gate implemented using four full-adders and two parallel adders. Each full-adder (Full-adder 1 to Full-adder 4) takes three inputs (A, B, and Cin) and produces three outputs (Σ, Cout, and t). The parallel adders take two inputs (A and B) and produce three outputs (Σ, S, and t). The outputs of the full-adders and parallel adders are connected to a 4-bit majority logic gate, which produces a 4-bit output.



BCD to excess 3 code converter

Think:

A code converter that will take 4 bit BCD code as input and generate the Excess 3 output using 4 bit parallel adder.

BCD to excess 3 code converter

Solution:

Inspection of truth table of 'excess 3 from BCD' shows that we can get excess-3 code from BCD by adding '0011' to each BCD number. This addition can be implemented by means of 4-bit full adder MSI circuit.

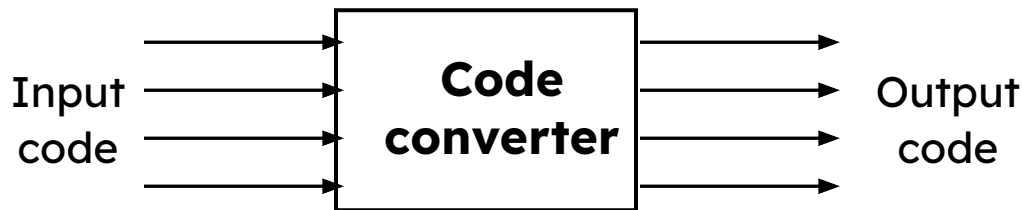
Let, ABCD as BCD input and WXYZ as Excess 3 output

Let, ABCD as BCD input and WXYZ as Excess 3 output



Code Converters

- Code converters – take an input code, translate to its equivalent output code.



- Example: **BCD to Excess-3 Code Converter.**

Input: BCD digit

Output: Excess-3 digit

Overflow Detection [Self Study]

- An overflow condition can be detected by observing the carry into the sign bit position (C3) and the carry out of the sign bit position (C4).
- If these **two carries are not equal**, an overflow has occurred.
- The two carries are applied to an exclusive-OR gate(XOR), an overflow is detected when the output(V) of the XOR gate is equal to 1.

Carry: 0 1	
+7	0 111
+6	0 110
+48	1 101

Overflow

Carry: 1 0	
-7	1 001
-6	1 010
-13	0 011

Overflow

Carry: 1 1	
-3	1 101
-2	1 110
-5	1 011

No Overflow

Carry: 0 0	
+2	0 010
- 4	1 100
-2	1 110

No Overflow

